

KinetAios

本地优先的多引擎 AI Agent 桌面应用

版本 1.5.0

协议 GPL v3

平台 Windows 11 / macOS

技术栈 Electron + TypeScript + SQLite (FTS5)

GitHub github.com/phinn/KinetAios

官网 phinn.github.io/KinetAios

KinetAios 项目运行说明

版本 1.5.0 · GPL v3 · Windows 11 / macOS

目录

- 1. 环境要求
- 2. 快速安装（推荐）
- 3. 从源码构建
- 4. 首次配置
- 5. 基本使用
- 6. Direct Engine 工具列表
- 7. 长期记忆系统
- 8. 插件系统
- 9. Ollama 本地模型配置
- 10. MCP 集成
- 11. 安全与沙箱
- 12. 常见问题

1. 环境要求

最低要求

项目	要求
操作系统	Windows 11 (64-bit) 或 macOS 12+
内存	4 GB（推荐 8 GB）
磁盘空间	300 MB（不含 Ollama 模型）
网络	安装时需联网（npm 依赖 + Electron 下载）

从源码构建额外需要

项目	要求
Node.js	18.0+ (推荐 20 LTS)
编译工具链	Windows: Visual Studio Build Tools / macOS: Xcode Command Line Tools
npm	随 Node.js 安装

2. 快速安装（推荐）

Windows

- 1. 前往 [GitHub Releases](#) 下载 `KinetAios-Setup-1.5.0.exe`
- 2. 双击运行安装程序
- 3. 如果出现 SmartScreen 警告 → 点击「更多信息」→「仍要运行」（应用未签名，这是正常的）
- 4. 安装完成后从开始菜单启动 KinetAios

macOS

- 1. 前往 [Releases](#) 下载 `.dmg` 文件
- 2. 打开 DMG，将 KinetAios 拖入 Applications 文件夹
- 3. 首次打开时 Gatekeeper 会警告 → 右键 →「打开」→ 确认

3. 从源码构建

3.1 克隆仓库

```
git clone https://github.com/phinn/KinetAios.git
cd KinetAios/KinetAiosWin
```

3.2 安装依赖

```
npm install
```

国内用户注意：如果 `npm install` 卡在下载 Electron 二进制，项目已预配 npmmirror 镜像（`.npmrc`）。如仍然失败，手动执行：

```
export ELECTRON_MIRROR=https://npmmirror.com/mirrors/electron/
node node_modules/electron/install.js
```

3.3 构建项目

```
npm run build
```

该命令会依次执行：

1. **tsc** — 编译主进程 + preload + shared 类型 (TypeScript → JavaScript)
2. **esbuild** — 打包渲染层 (Vanilla TS → 单文件 bundle)
3. 复制资源 — **brand.json**、HTML、CSS 等静态文件到 **dist/**

3.4 启动应用

```
npm start
```

或一步到位（构建 + 启动）：

```
npm run dev
```

3.5 打包安装程序

```
# Windows (必须在 Windows 机器上执行)
npm run dist
# → 产出 release/KinetAios Setup 1.5.0.exe

# 或只打包不制作安装程序
npm run pack
# → 产出 release/win-unpacked/KinetAios.exe
```

macOS 用户注意：无法从 macOS 构建 Windows 安装包（native 模块 **better-sqlite3** 跨平台编译不可靠）。请使用 GitHub Actions **windows-latest** runner 或在 Windows 机器上构建。

3.6 验证构建

```
npm run typecheck
```

如果没有任何输出，说明类型检查通过，构建成功。

4. 首次配置

4.1 打开设置

启动应用后，点击窗口右上角的 ⚙️ 齿轮图标。

4.2 配置 LLM 接口

字段	说明	示例
API Key	LLM 提供商 API 密钥	XXXXXXXX.XXXXXXXX
Base URL	API 端点地址（留空走默认）	https://open.bigmodel.cn/api/paas/v4/
模型	要调用的模型 ID	glm-4-plus、deepseek-chat

预设提供商

提供商	Base URL	默认模型
GLM（智谱）	https://open.bigmodel.cn/api/paas/v4/	glm-4-plus
DeepSeek	https://api.deepseek.com/v1/	deepseek-chat
OpenAI	https://api.openai.com/v1/	gpt-4o
Anthropic	协议切为 anthropic	claude-3-5-sonnet

4.3 测试连接

点击「测试连接」按钮。如果返回成功提示，说明配置正确，可以开始使用。

4.4 模型配置档 (Profiles)

在设置页可以创建多个「模型配置档」，每个会话可独立选择不同的配置档：

- Profile A：GLM-4-Plus（日常任务）
- Profile B：DeepSeek-Chat（代码任务）
- Profile C：Ollama llama3.2（离线任务）

4.5 API Key 加密存储

API Key 使用操作系统原生安全存储机制加密：

- macOS — Keychain
- Windows — DPAPI（Data Protection API）

5. 基本使用

5.1 创建会话

点击侧边栏 + 按钮创建新会话。每个会话拥有独立的引擎、模型、工作目录和对话历史。

5.2 发送任务

在底部输入框输入任务描述，按 `Enter` 发送。Direct Engine 会：

- 1. 将消息发送给 LLM
- 2. 如果 LLM 返回工具调用（`tool_calls`），执行对应工具
- 3. 将工具结果拼回上下文，再次请求 LLM
- 4. 循环直到 LLM 返回纯文本回答（无工具调用）

5.3 工具调用确认

当 Agent 请求执行 `shell` 命令时，会弹出确认窗口。可在设置中调整审批策略：

策略	说明
<code>always</code>	每次都确认（默认，最安全）
<code>never</code>	永不确认（全自动，需自行承担风险）

5.4 切换引擎

在会话顶部下拉菜单切换引擎。切换会清除跨引擎上下文（但长期记忆不受影响）。

5.5 全局热键

`Ctrl/Cmd + Alt + Space` 从任何应用唤出快速面板。

6. Direct Engine 工具列表

Direct Engine 内置 12 个工具，Agent 在推理过程中自动选择调用：

工具	功能	安全措施
shell	执行任意 shell 命令	用户确认 + 审批策略
read_file	读取 UTF-8 文本文件	readOnly 沙箱限制在 cwd 内
write_file	写入文件（覆盖）	写入前自动快照 + workspaceWrite
edit_file	精确查找替换	写入前自动快照 + 严格匹配
grep	递归正则搜索文件内容	自动排除 node_modules/.git
glob	按模式列出文件	只读
git_diff	查看 git 改动	只读
web_search	多引擎搜索	Bing + DuckDuckGo 回退
web_fetch	抓取 URL 并去噪	SSRF 防护 + Jina Reader
recall_memory	搜索长期记忆	FTS5 + embedding 语义召回
dispatch_agent	派发只读子 Agent	子 Agent 无 shell/write 权限

文件快照回滚

每次 `write_file` 或 `edit_file` 执行前，原文件内容会被保存到 `.kinet-snapshots/` 目录。快照上限 200 条，超过自动删最旧的。用户可一键回滚到任意快照版本。

7. 长期记忆系统

7.1 自动记忆提取

每轮对话结束后，后台自动分析内容，提取「关于用户的持久事实」→ 存入 SQLite + FTS5。

7.2 跨会话注入

下一次对话开始时（即使换了模型或引擎），相关记忆自动注入 system prompt。

7.3 记忆面板

- 查看所有记忆（按频道/全局分类）
- 行内编辑或删除单条记忆
- 查看每条记忆的来源会话和提取时间

- 记忆导入/导出 JSON

7.4 记忆图谱

力导向图可视化：节点 = 记忆条目，边 = 语义关联，**红色标记** = 冲突记忆。

8. 插件系统

8.1 安装插件

将插件文件夹放入 `userData/plugins/` 目录，重启应用自动加载。

平台	路径
Windows	<code>C:\Users\用户名\AppData\Roaming\KinetAios\plugins\</code>
macOS	<code>~/Library/Application Support/KinetAios/plugins/</code>

8.2 内置插件

插件	功能	工具数
office-suite	Excel 读写、PDF 生成、OCR、Word 操作、文档格式互转	18
brainstorm	Excalidraw 思维导图	—
math-practice	数学练习	—
cpp-learning	C++ 学习模块	—

8.3 插件能力

- **工具 (Tools)** — Agent 可调用的新函数
- **Slash 命令** — 输入 `/` 触发的快捷命令
- **全屏面板** — 独立的 UI 界面
- **System Prompt 注入** — 按关键词匹配，按需注入 (省 ~60% token)

9. Ollama 本地模型配置

9.1 安装 Ollama

前往 ollama.com 下载并安装。

9.2 拉取模型

```
ollama pull llama3.2    # 或 qwen2.5, glm4 等
```


9.3 在 KinetAios 中配置

KinetAios 会自动检测本地 Ollama (`localhost:11434`):

1. 在设置页创建新的模型配置档
2. Base URL 填入 `http://localhost:11434`
3. 模型名填入 Ollama 中的模型名 (如 `llama3.2`)
4. API Key 填任意值 (Ollama 不需要认证)
5. KinetAios 自动切换到 Ollama 原生 `/api/chat` 协议, 设置 32K 上下文

9.4 完全离线

配置 Ollama 后, KinetAios 可以在完全断网的环境下运行 (Agent 推理 + 工具执行 + 本地记忆)。

10. MCP 集成

10.1 MCP 客户端

KinetAios 自动扫描系统 MCP 服务配置 (`~/.claude.json` 、 `~/.codex/config.toml` 、 Claude Desktop 配置)。通过 stdio 连接, 断开自动重连。

10.2 MCP 服务端

内置 MCP Server (HTTP + SSE), 暴露 `run_agent` 工具:

1. 在设置中启用 MCP Server, 获取 Token
2. 其他机器用标准 MCP 客户端连接
3. 调用 `run_agent` 远程触发本机 Agent 执行任务

安全措施: Token 鉴权 (恒定时间比较)、5 分钟超时、僵尸连接自动清理。

11. 安全与沙箱

11.1 沙箱模式

模式	说明
<code>readOnly</code>	Agent 只能读取工作目录内文件, 禁止写操作
<code>workspaceWrite</code>	Agent 可读写工作目录内文件, 不能写目录外
<code>fullAccess</code>	不限制 (默认)

11.2 SSRF 防护

`web_fetch` 工具内置 SSRF 防护:

- 拦截 IPv4 私有段 (10/8、172.16/12、192.168/16)
- 拦截 loopback (127/8)、link-local (169.254/16, 含云元数据端点)
- 拦截 CGNAT (100.64/10)
- 拦截 IPv6 loopback / ULA
- DNS 解析后二次检查实际 IP (防 DNS rebinding)

11.3 错误脱敏

返回给 LLM 的错误信息自动去掉绝对路径、堆栈跟踪，截断到 300 字符。

11.4 Shell 确认桥

Agent 请求执行 shell 命令时，通过 IPC 弹出模态窗口，用户可查看完整命令并批准或拒绝。

12. 常见问题

Q: 启动后看不到界面？

A: 检查任务栏/托盘。KinetAios 默认最小化到托盘。使用 `Ctrl+Alt+Space` 唤出快速面板。

Q: npm install 报错 (better-sqlite3 编译失败) ？

A: 确认已安装编译工具链：Windows 安装 Visual Studio Build Tools；macOS 执行 `xcode-select --install`。

Q: API Key 安全吗？

A: API Key 使用操作系统原生加密存储（macOS Keychain / Windows DPAPI），不以明文写入磁盘。

Q: 多个会话会互相影响吗？

A: 不会。每个会话有独立的上下文、历史和工具执行环境。唯一共享的是长期记忆。

Q: 如何切换语言？

A: 设置 → 界面 → 语言。支持 English / 简体中文 / 繁體中文 / 日本語。

Q: 支持 Linux 吗？

A: 目前官方支持 Windows 11 和 macOS。Linux 理论可行（Electron 跨平台），但未正式测试。

附录：常用命令速查

命令	说明
<code>npm install</code>	安装依赖 + 重编 better-sqlite3
<code>npm run build</code>	构建项目（tsc + esbuild + 复制资源）
<code>npm run typecheck</code>	类型检查（不产出文件）
<code>npm start</code>	启动应用（需先 build）
<code>npm run dev</code>	构建 + 启动
<code>npm run pack</code>	打包（不制作安装程序）
<code>npm run dist</code>	制作安装程序（NSIS / DMG）

附录：键盘快捷键

快捷键	功能
<code>Ctrl/Cmd + Alt + Space</code>	全局热键，唤出快速面板
<code>Ctrl/Cmd + K</code>	全局搜索（搜索所有会话）
<code>Ctrl/Cmd + Shift + I</code>	打开 DevTools
<code>Enter</code>	发送消息
<code>Shift + Enter</code>	换行（不发送）